

2.1

(1) 程序代码:

```
#include <stdio.h>
#include <stdlib.h>

// data 段
int initialized_data = 42;
// bss 段
int uninitialized_data;

int main()
{
    // 动态分配内存 (堆)
    int *heap_data = (int *)malloc(sizeof(int));
    if (heap_data == NULL)
    {
        fprintf(stderr, "内存分配失败\n");
        return 1;
    }
    // 使用分配的内存
    *heap_data = 100;
    // 打印变量的值
    printf("initialized_data: %d\n", initialized_data);
    printf("uninitialized_data: %d\n", uninitialized_data);
    printf("heap_data: %d\n", *heap_data);
    // 释放分配的内存
    free(heap_data);
    return 0;
}
```

data 段: initialized_data

bss 段: uninitialized_data

堆: 无

(2) readelf:

[25]	.data	PROGBITS	00000000000004000	00003000
	00000000000000014	0000000000000000	WA	0 0 8
[26]	.bss	NOBITS	00000000000004020	00003014
	00000000000000010	0000000000000000	WA	0 0 32

objdump:

```
Contents of section .data:
4000 00000000 00000000 08400000 00000000 .....@.....
4010 2a000000                                     *...
```

(3)(1)中所写程序用到了栈, 具体来说, 函数 main 中局部变量就存储在栈上。

2.2

(1) C 程序:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int arr[10];
    for (int i = 0; i < 10; i++)
    {
        arr[i] = i + 1;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        fprintf(stderr, "Fork failed\n");
        return 1;
    }
    else if (pid == 0)
    {
        // 子进程
        int sum = 0;
        for (int i = 0; i < 10; i++)
        {
            sum += arr[i];
        }
        printf("Sum of array elements: %d\n", sum);
        exit(0);
    }
    else
    {
        // 父进程
        wait(NULL); // 等待子进程完成
        printf("parent process finishes\n");
    }

    return 0;
}
```

```
}
```

打印结果:

```
Sum of array elements: 55
parent process finishes
```

说明:首先创建一个包含 1 到 10 的整数数组,然后使用 `fork()` 创建一个子进程,而后子进程完成计算并打印结果,最终父进程打印“parent process finishes”,再退出。

(2) C 程序:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int arr[10];
    for (int i = 0; i < 10; i++)
    {
        arr[i] = i + 1;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        fprintf(stderr, "Fork failed\n");
        return 1;
    }
    else if (pid == 0)
    {
        // 子进程
        int sum = 0;
        for (int i = 0; i < 10; i++)
        {
            sum += arr[i];
        }
        printf("Sum of array elements: %d\n", sum);

        // 使用 system() 函数执行 ls -l 命令
        int ret = system("ls -l");
    }
}
```

```

        exit(0);
    }
    else
    {
        // 父进程
        wait(NULL); // 等待子进程完成
        printf("parent process finishes\n");
    }

    return 0;
}

```

打印结果：

```

Sum of array elements: 55
total 28
-rw-r--r-- 1 zeriwang zeriwang 622 Sep 3 20:08 2_1_1.c
-rw-r--r-- 1 zeriwang zeriwang 682 Sep 4 15:36 2_2_1.c
-rwxr-xr-x 1 zeriwang zeriwang 16304 Sep 4 15:54 2_2_2
-rw-r--r-- 1 zeriwang zeriwang 780 Sep 4 15:54 2_2_2.c
parent process finishes

```

说明:与(1)中程序类似,子进程在完成运算和打印后,执行 `ls -l` 命令再退出。

(3)

进程控制块 (PCB) 的代码在 `kernel` 文件夹下的 `proc.c` 文件中,在这个文件中的数据结构 `struct proc` 定义了进程控制块。

当 `fork` 函数执行时,它会对 PCB 进行以下操作:

1. 分配新进程的 `proc` 结构体。
2. 将父进程的上下文(包括寄存器状态、页表等)复制到新进程中。这包括复制父进程的内存空间、文件描述符等。
3. 设置新进程的标识信息
4. 初始化新进程的内核栈
5. 设置新进程的状态

2.3

(1) 3 个子进程,关系图如下:

主进程 (A)

```

|—— 子进程 (B) - 第一次循环创建
|   |—— 子进程 (D) - 第二次循环创建
|   |—— 子进程 (C) - 第二次循环创建

```

说明：

第一次循环（loop = 0）中，主进程（A）调用 fork()，创建一个子进程（B）；
第二次循环（loop = 1）中，主进程（A）调用 fork()，创建一个子进程（C），子进程（B）调用 fork()，创建一个子进程（D）。

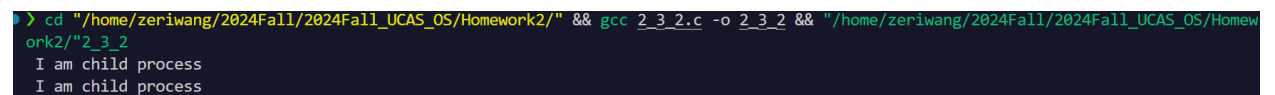
（2）修改后代码：

```
#include<unistd.h>
#include<stdio.h>
#include<string.h>
#define LOOP 2

int main(int argc, char *argv[])
{
    pid_t pid;
    int loop;

    for(loop=0; loop<LOOP; loop++) {
        if((pid=fork()) > 0)
            sleep(5);
        else if(pid == 0) {
            printf(" I am child process\n");
            // 结束子进程
            return 0;
        }
        else {
            fprintf(stderr, "fork failed\n");
        }
    }
    return 0;
}
```

结果截图：



```
> cd "/home/zeriwang/2024Fall/2024Fall_UCAS_OS/Homework2/" && gcc 2_3_2.c -o 2_3_2 && "/home/zeriwang/2024Fall/2024Fall_UCAS_OS/Homework2/"2_3_2
I am child process
I am child process
```

说明：

修改后代码的 for 循环中，当 pid = 0 即当前进程为子进程时，在打印 I am child process 后，就会退出，进而不会生成额外的子进程，最终达成序生成 LOOP 个子进程的效果